LAB REPORT: WORKSHOP 3

LOCAL DEPLOYMENT OF AN INTERACTIVE WEBSITE USING APACHE2, HTML, CSS AND JQUERY
WEB APPLICATIONS COURSE

Kevin Sanchez
Yachay Tech University

Wednesday 9th September, 2026

Abstract

This report documents the implementation of Workshop 3 of the Web Applications course, which extends a semantic HTML5 / CSS3 personal portfolio (built in Workshops 1 and 2) with client-side interactivity using jQuery. The site is served locally through an Apache2 HTTP server running on Ubuntu under WSL2, resolved under the custom domain `workshop1.webapp`. In this work we add three interactive features: a random hero background color toggle, a popup image modal for the profile photo, and full client-side validation of the contact form. The report describes the tools and architecture used, the two main technical obstacles encountered during development, and includes responsive screenshots (mobile, tablet and desktop) of the resulting website.

1 Introduction

The purpose of this workshop is to add dynamic, client-side behavior to a static website using **jQuery** library of JavaScript programming language, while keeping the underlying deployment infrastructure built in the previous workshops: a semantic HTML5 structure (Workshop 1) served through a locally configured **Apache2** server, and a fully responsive with media queries using CSS3 for design the system (Workshop 2).

2 Project Structure and Tools

The project is organized as a single Git repository with one folder per workshop, so that the evolution of the portfolio can be tracked incrementally:

```
Web-applications-course-Yachay-Tech/
|-- workshop-1-website-HTML/ (Workshop 1: semantic HTML + Apache2)
|-- workshop-2-website-HTML-with-CSS/(Workshop 2: CSS3 responsive design)
|-- workshop-3-website-with-JS/ (Workshop 3: JavaScript + jQuery)
    |-- index.html
    |-- js/main.js (hero color toggle + image modal)
    |-- js/form_validation.js (contact form validation)
    |-- css/ (styles, including validation states)
    |-- pages/ (about, professional, extracurricular,
                                courses, schedule, contact)
```

The tools used throughout the workshop are:

- **HTML5** for the semantic page structure (`<header>`, `<nav>`, `<main>`, `<section>`, `<form>`).
- **CSS3** (custom properties, Flexbox and CSS Grid) for the visual design and the three responsive breakpoints (mobile $<$ 768px, tablet 768–1024px, desktop $>$ 1024px).
- **jQuery** (loaded via CDN) for DOM selection and event handling.
- **Apache2** on Ubuntu (WSL2) as the local HTTP server, exposed under the custom hostname `workshop1.webapp`.
- **Git / GitHub** for version control.

3 Implementation

The project was built incrementally, layer by layer, following the same order in which a browser actually processes a page. The starting point was the deployment infrastructure inherited from Workshop 1: the Apache2 Virtual Host for `workshop1.webapp` was kept as is, with its `DocumentRoot` pointing to the repository folder mounted from the Windows filesystem into WSL2, the hostname resolved to `127.0.0.1` through entries in both the Windows and the Linux `/etc/hosts` files, and WSL2's mirrored networking mode enabled in `.wslconfig` so that the Windows browser and the Apache daemon running inside the Linux VM share the same network namespace. With the server already serving files, the next step was to build the HTML skeleton: each page of the portfolio (home, about, professional, extracurricular, courses, schedule and contact) was structured using semantic HTML5 elements such as `<header>`, `<nav>`, `<main>`, `<section>`, `<table>` and `<form>`, with no CSS involved yet, so that the document would remain readable and navigable on its own.

Once the skeleton was in place, the presentation layer was added on top of it. A design system based on CSS custom properties (colors, spacing and variables defined in `main_styles.css`) was used to keep every page-specific stylesheet consistent, while Flexbox and CSS Grid handled the layout of components such as the navigation bar, the card grids and the weekly schedule timetable. Only after the desktop layout looked correct were responsive media queries introduced, splitting the design into three breakpoints (mobile (< 768px), tablet (768–1024px) and desktop (> 1024px)), so that the navigation bar collapses into a horizontal scrollable strip, the card grids reflow from a single column to two or three columns, and the call-to-action buttons stack full-width on small screens instead of sitting side by side.

With the structure and the styling finished, the last layer added was client-side behavior, implemented with jQuery on top of the already-styled markup. Three independent interactions were wired in `js/main.js` and `js/form_validation.js`. First, a button in the hero section listens for a `click` event and, on every click, selects a random color from a predefined `colorPalette` array using `Math.random()` and applies it to the hero's background through `.css()`. Second, clicking the profile photo enlarges it inside a popup modal that sits hidden in the DOM. Third, the contact form's `submit` event is intercepted, and the majority of fields can be validated on the client before anything would normally be sent, for example, regular expressions check the name and the email, simple non-empty checks cover the remaining text and select fields, and a required checkbox enforces consent, if any field that fails is marked with an `.input-error` class and an inline error message, a live listener clears that state as soon as the user corrects the value, and once every field passes, an `alert()` confirms that the form was submitted successfully.

4 Challenges Faced

4.1 Configuring the Apache2 server on WSL2

The first obstacle was not JavaScript-related but infrastructural: getting the Windows browser to actually reach the Apache2 daemon running inside WSL2 under a custom hostname. After mapping `workshop1.webapp` to `127.0.0.1` in both the Windows and the WSL `hosts` files, the browser still failed to resolve the name, because by default Windows issued the request over IPv6 while WSL2's NAT-based networking only forwarded IPv4 traffic on `localhost`. This was solved by switching WSL2 to *mirrored* networking mode through a `.wslconfig` file, which makes the WSL2 VM share the host's network interfaces directly instead of living behind a separate virtual NAT adapter. A second, related issue appeared once the Virtual Host was enabled: Apache returned an `AH01630: client denied by server configuration` error, because defining `DocumentRoot` inside `<VirtualHost>` does not by itself grant filesystem access permissions when the document root lives on a path mounted from the Windows drive (`/mnt/c/...`). The fix was to add an explicit `<Directory>` block with `Require all granted` for that path, after which `apache2ctl configtest` reported `Syntax OK` and the site loaded correctly.

4.2 JavaScript event handlers executing on page load instead of on the event

The second challenge appeared while wiring the jQuery event handlers for the color toggle and the modal. The initial code looked like:

```
$boton.on("click", changeColor()); // WRONG
```

Because `changeColor()` was written *with* parentheses, JavaScript evaluated the function call immediately while the script was being parsed, and passed its *return value* (typically `undefined`) as the second argument to `.on()`. As a result, the color change ran exactly once on every page reload, instead of running each time

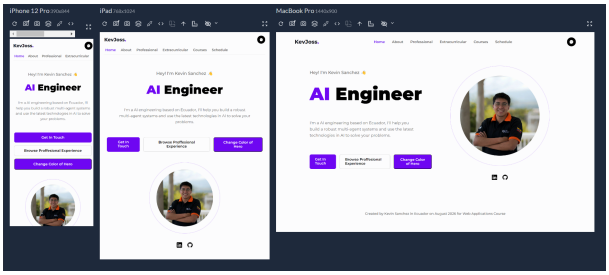
the button was clicked. The fix was to pass a *reference* to the function, without invoking it, so that jQuery itself calls it later whenever the event fires:

```
$boton.on("click", changeColor); // CORRECT
```

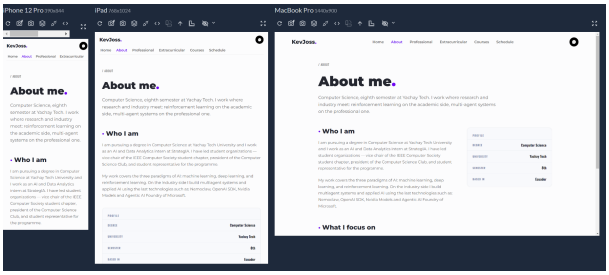
The same mistake initially affected the modal's open/close handlers and the form's submit handler, and was corrected the same way. This reinforced an important distinction in JavaScript: `functionName` refers to the function itself (a value that can be stored and called later), while `functionName()` invokes it right away.

5 Results

Figures 1(a)–4(a) show the six main pages of the portfolio rendered at three viewport widths (iPhone 12 Pro, iPad, and MacBook Pro), confirming that the CSS Grid timetable, the navigation bar and the contact form all adapt correctly across breakpoints. Figures 5, 4(b) and 6 demonstrate the three jQuery features specifically requested for this workshop.

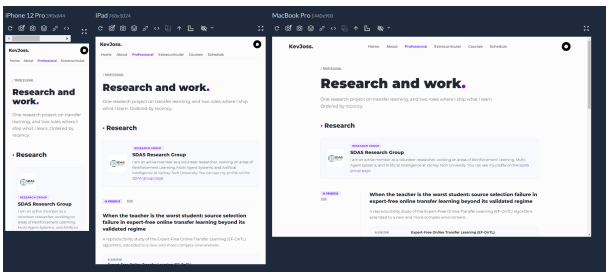


(a) Home

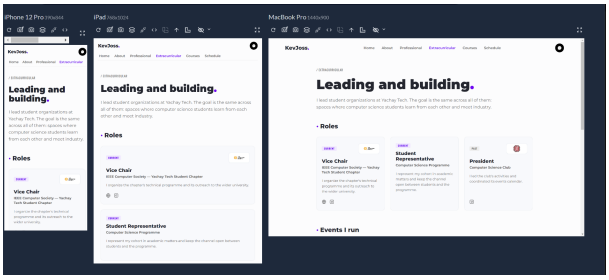


(b) About

Figure 1: Home and About pages across the three responsive breakpoints (iPhone 12 Pro, iPad, MacBook Pro). The hero's call-to-action buttons stack vertically on mobile and sit side by side on desktop; the About page's profile metadata card moves from a stacked list to a side card.

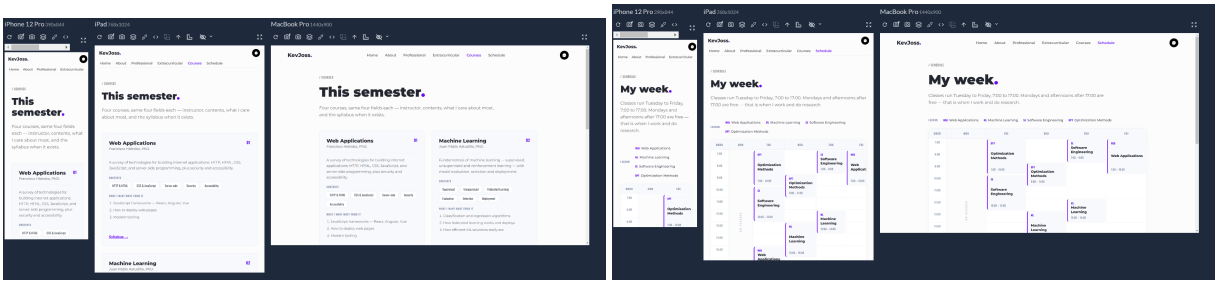


(a) Professional



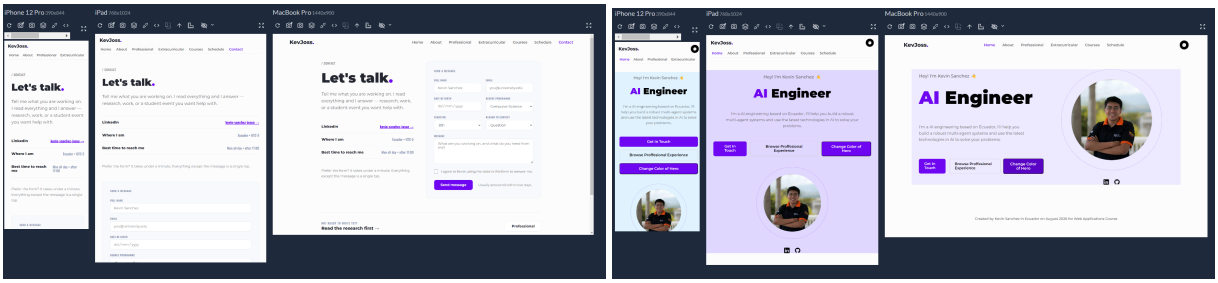
(b) Extracurricular

Figure 2: Professional page (SDAS Research Group and the ongoing reproducibility study) and Extracurricular page (current and past leadership roles), with cards reflowing from one to three columns across breakpoints.



(a) Courses (b) Schedule

Figure 3: Courses page listing the four courses of the semester, and the weekly Schedule page implemented with CSS Grid, with all cell collisions resolved across breakpoints.



(a) Contact (b) Hero color toggle

Figure 4: (a) Contact page with the form used for client-side validation. (b) Result of clicking "Change Color of Hero": the background changes to a random palette color while the profile photo ring stays visible, confirming the corrected `z-index` stacking.

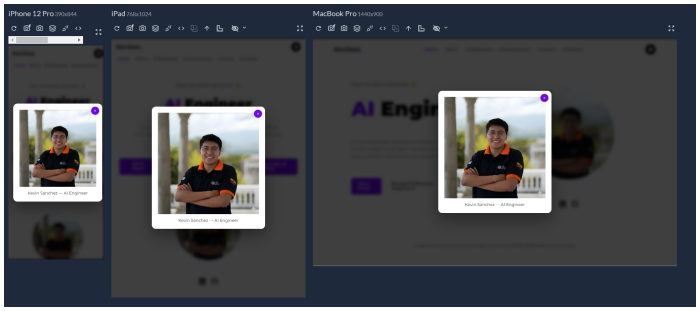
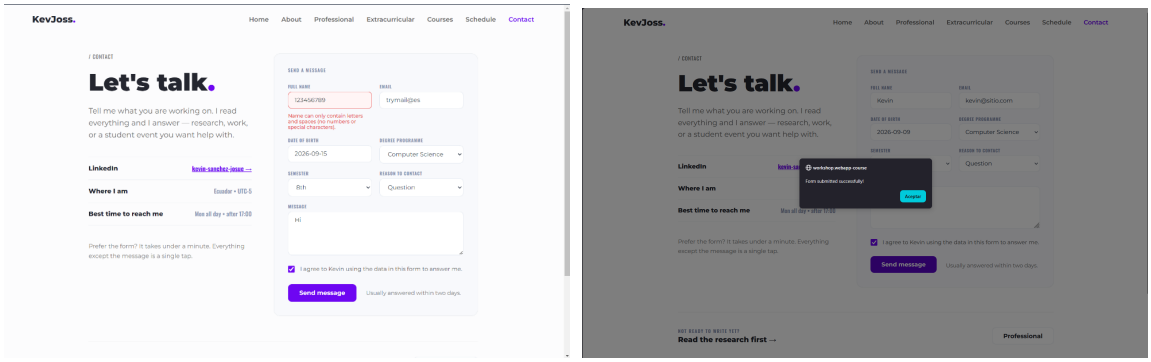


Figure 5: Image modal opened by clicking the profile photo, shown across the three breakpoints with the dark overlay and the close button.



(a) Invalid name field

(b) Successful submission

Figure 6: Client-side form validation. (a) Submitting a numeric value in the "Full Name" field is rejected inline with a red border and an error message, without leaving the page. (b) Once every field is valid, the browser shows the `alert()` confirming "Form submitted successfully!".

6 Conclusion

The workshop's requirements "a navigation bar, hero section, image gallery/modal and a validated contact form, all made interactive with jQuery", were met on top of the Apache2/WSL2 deployment built in the previous workshops. Both challenges encountered, one at the infrastructure level (WSL2 networking and Apache directory permissions) and one at the code level (passing a function reference instead of invoking it inside `.on()`). The resulting site correctly reflows across mobile, tablet and desktop breakpoints for all six pages, and all three required jQuery behaviors (hero color toggle, image modal, form validation) work as specified.

7 Repository

The full source code, including Workshops 1, 2 and 3, is available at:
`https://github.com/KevJoss/Web-applications-course-Yachay-Tech`